11 | 基础组件（下）：前端 UI 设计与前端基础组件

Robert · Claude Code 企业级全链路开发实战



你好，我是 Robert。

上一讲把后端基础组件全部搭完了，三档就绪。这一讲搭前端。

09 讲搭了前端空壳，包括 Vue 骨架、axios 封装、三个菜单页面的占位文字。能跑，但说实话，看起来就是 Element Plus 的默认灰蓝，像一个课程作业，不像一个产品。

这一讲做两件事：

1. 让 Claude Code 当设计师，把界面从“能用”变成“好看”；
2. 把前端业务开发需要的公共组件一次性封装好。

做完之后，前端也和后端一样，基础设施就绪，下个章节开始直接往里填业务功能。

让 AI 当设计师

大多数后端工程师没有设计师的审美，不知道配色怎么选、间距怎么给、什么风格适合什么产品。以前这事只能找设计师或者产品经理，现在 Claude Code 能帮你做到 80 分以上。

但前提和写代码一样：你的输入越具体，它的输出越好。不是说“帮我设计一个好看的界面”，而是要告诉它产品是什么、用户是谁、你想要什么风格。01 讲说的“思考质量决定输出质量”，在 UI 设计这个场景同样适用。

第一步：定风格方向

先用咨询模式给 Claude Code 足够的上下文：

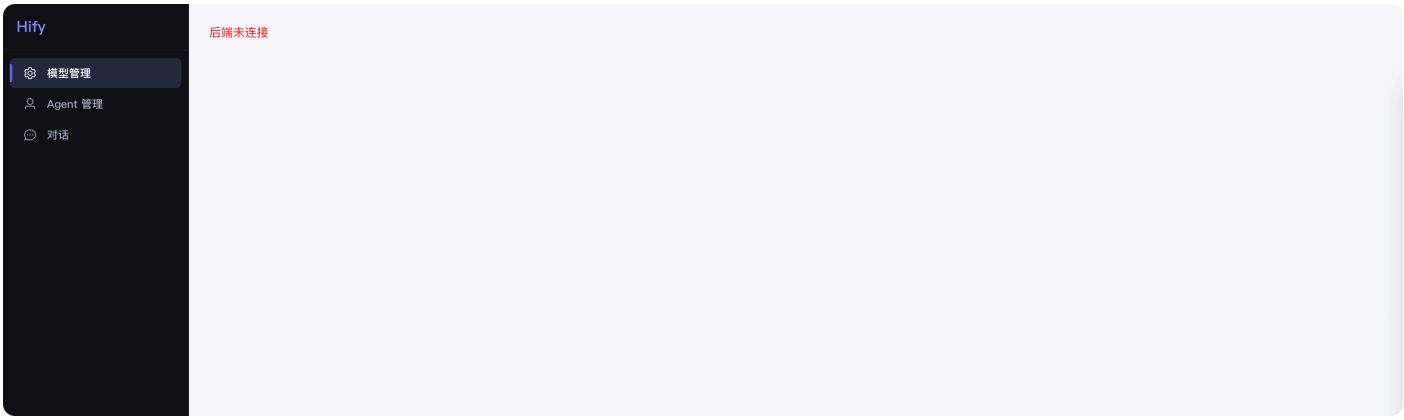
> Hify 是一个 AI Agent 开发平台，面向技术团队内部使用，主要用户是开发者和技术管理者。界面以管理后台为主——大量的表格、表单、配置页面，加上一个对话交互页面。我想要的视觉风格：浅底 + 科技感点缀。整体用浅色背景保持信息可读性（管理后台表格多，深色底长时间看眼睛累）。但不要太素——侧边栏用深色底，按钮和关键交互元素用亮色，制造科技感和品牌感。色调方向：主色用蓝紫系（科技感强），辅色用青色或薄荷绿（数据 / 状态指示）。参考 Linear、Supabase 的视觉风格——干净但不无聊，有设计感但不花哨。帮我设计一套完整的设计系统：主色 / 辅色 / 背景色阶 / 文字色阶 / 圆角 / 阴影 / 过渡动效，用 CSS 变量输出。

注意这条指令的结构：先说产品是什么（上下文），再说用户是谁（约束），再说我要什么风格（方向），再说具体的色调偏好（细节），最后给参考（锚点）。层层递进，Claude Code 不需要猜。

命令执行后遇到一个问题：



这说明了 Claude Code 不是能一下子完成任务的，而是需要不断调整的。经过调整后，效果如下：



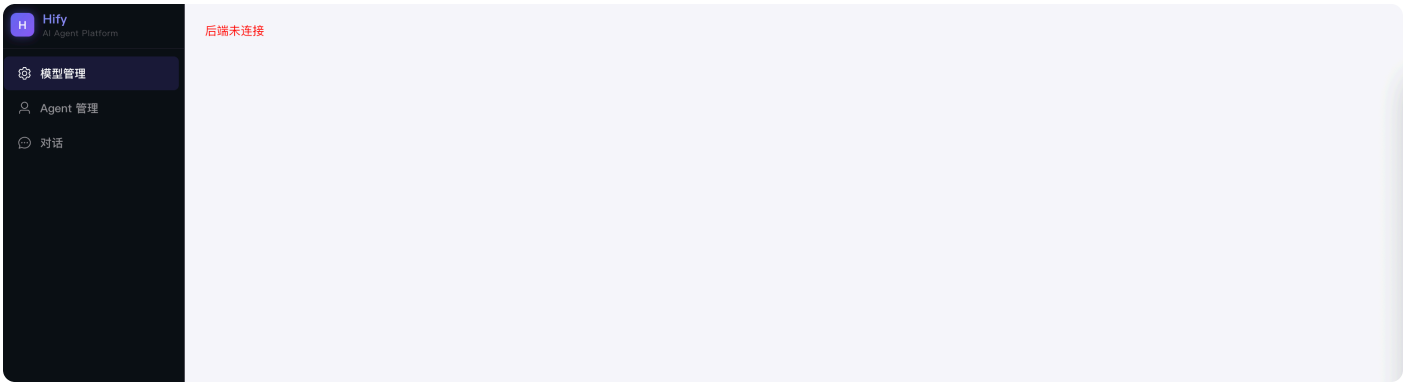
拿到这套变量后，快速 review 一下：颜色搭配刺不刺眼、对比度够不够（文字要能看清）、整体调性对不对。不需要逐个色值较真，大方向对就行，细节后面调。

第二步：改造侧边栏

侧边栏是用户打开页面第一眼看到的东西，改造效果最立竿见影。

> 改造 Hify 的侧边栏。要求：
> 1. 背景用深色（接近纯黑但不是纯黑，用 --color-bg-dark），和浅色内容区形成对比
> 2. 顶部 Logo 区域：显示"Hify"品牌名，用主色渐变文字，下面一行小字“AI Agent Platform”
> 3. 菜单项：默认白色文字 + 透明背景，hover 时背景微亮（rgba 白色 10% 透明度），选中态左边一条 3px 的主色竖线 + 背景微亮
> 4. 菜单图标：每个菜单项前加 Element Plus 图标（模型管理用 Setting，Agent 管理用 User，对话用 ChatDotRound）
> 5. 底部：折叠 / 展开按钮，版本号
> 6. 整体要炫酷，有科技感，但不能花哨到影响使用

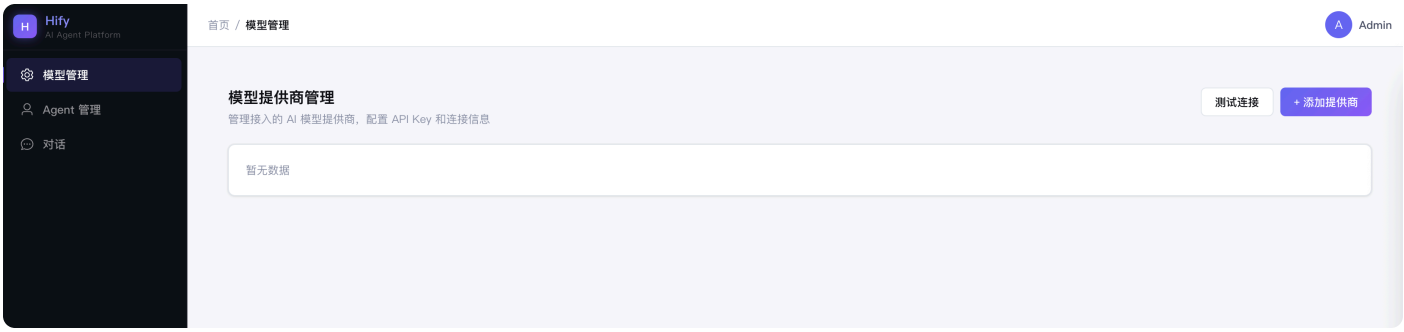
效果如下：



第三步：页面整体布局

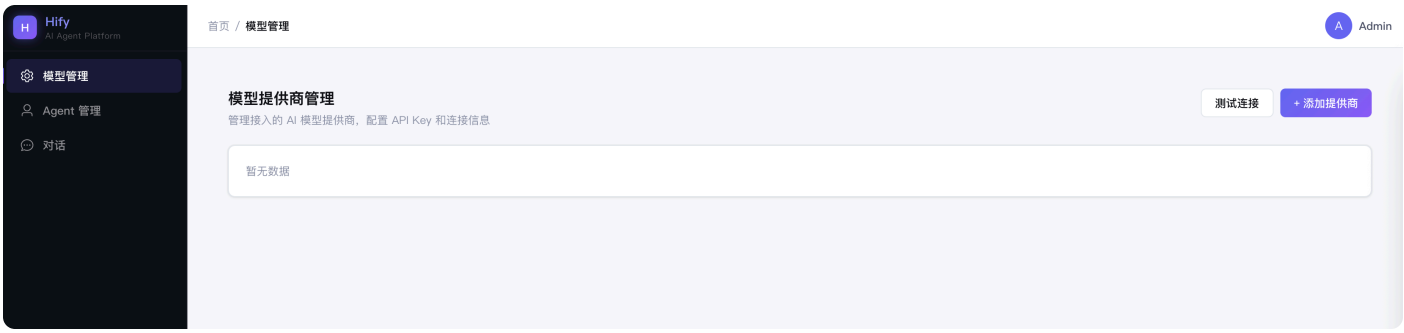
> 优化 Hify 的页面整体布局：
> 1. 顶栏：左侧面包屑导航（显示当前页面路径），右侧用户信息区域（头像 + 用户名 placeholder）
> 2. 内容区：背景用 --color-bg-secondary（浅灰），内容卡片用白色背景 + 轻阴影 + 圆角，和背景有层次感
> 3. 页面标题区域：每个页面顶部有标题 + 描述文字 + 操作按钮区（右侧）
> 4. 间距统一：页面边距 24px，卡片内边距 20px，元素间距 16px
> 5. 按钮样式：主要操作用主色渐变按钮（蓝紫渐变），次要操作用白色边框按钮，危险操作用红色

效果如下：



效果对比

到这里，UI 改造基本完成。



同一个页面结构，只是换了设计系统——CSS 变量 + 侧边栏 + 布局优化，没有改任何功能代码，但视觉感受完全不同。

这就是让 AI 当设计师的价值：你不需要懂设计理论，只需要能描述你想要什么。“浅底科技感、侧边栏深色、蓝紫主色、参考 Linear”，这些描述任何一个后端工程师都能给出来，Claude Code 会把它翻译成具体的 CSS。

前端基础组件：一条指令搞定

后端工程师做前端，最怕的不是写不出来，是写得慢。每个页面重复处理表格分页、弹窗打开关闭、loading 状态、删除确认，把这些封装成公共组件，后面做业务页面就是填配置。

我们是后端工程师，不需要在前端组件的实现细节上花太多时间。用一条完整的指令让 Claude Code 一次性生成所有组件：

> 在 hify-web 中创建以下前端公共组件，放在 src/components/ 目录下。所有组件使用 Vue 3 Composition API + TypeScript + Element Plus：
> 1. HifyTable.vue：通用列表页表格组件。Props 接收 columns 配置（label/prop/width/slot）、api 方法（返回 PageResult 格式）、是否显示分页。内部自动管理 loading 状态、分页参数、数据请求。暴露 refresh() 方法供外部调用刷新。空状态显 Element Plus 的 el-empty。
> 2. HifyFormDialog.vue：通用表单弹窗组件。Props 接收 title、width、表单 rules。v-model 控制显示隐藏。内部管理提交 loading、关闭时自动重置表单。暴露 open(data?) 方法，传 data 为编辑模式，不传为新增模式。提交时触发 submi 事件，由父组件处理 API 调用。

3. src/composables/useConfirm.ts：删除确认 composable。接收确认文案和 API 方法，弹出 ElMessageBox 确认框，确认后调用 API，成功后显示 ElMessage 成功提示，返回 Promise。一行代码完成 “确认删除→调接口→提示成功” 全流程。
4. src/composables/useRequest.ts：请求状态管理 composable。接收 API 方法，返回 { data, loading, error, execute }。自动管理三态，避免每个页面写 try-catch-finally 样板代码。
5. src/utils/notify.ts：统一通知封装。导出 notifySuccess/notifyError/notifyWarning 三个方法，底层调用 ElMessage，统一配置 duration 和样式。
   每个组件要有 TypeScript 类型定义，用泛型支持不同数据类型。组件风格和第一步定的设计系统一致。

一条指令，五个组件。Claude Code 生成后，快速过一遍：TypeScript 类型对不对、组件接口设计合不合理、和 Element Plus 的集成有没有问题。前端组件的实现细节不需要逐行 review，能用就行，有问题后面调。

这就是 01 讲说的三层分工里的第二层——AI 做，你验收。前端组件封装不是你的核心能力区，让 AI 高效产出，你把精力放在验证它好不好用上。这块就不展开了。如果有兴趣你可以去深究一下细节。

细节打磨：和 Claude Code 来回调

UI 设计和组件都有了，但实际用起来一定会有不满意的地方，比如间距不对、对齐有问题、颜色偏了、表格太挤、弹窗太宽。这是最花时间的环节，也是最能体现人机协作价值的环节。

我用 ProviderList 页面来演示这个过程。先让 Claude Code 生成一个完整的页面：

> 用 HifyTable 和 HifyFormDialog 实现 Provider 列表页面（src/views/provider/ProviderList.vue）。用 mock 数据，不调真实 API。
> 列表展示：名称、类型（OpenAI/Claude/Gemini/Ollama）、Base URL、状态（启用 / 禁用，用 el-tag 显示）、创建时间、操作（编辑 / 删除）。
> 页面顶部有标题 “模型提供商管理”、描述文字、右侧 “新增提供商” 按钮。
> 点新增弹出 HifyFormDialog，表单字段：名称、类型（下拉）、API Key、Base URL。

> 删除用 useConfirm。
> mock 数据写 5 条，类型分布开。

Claude Code 生成后，启动前端看效果。大概率第一版有这些问题，这是正常的，不是 Claude Code 能力不够，是 UI 细节需要人来判断。

1. 调布局

> ProviderList 页面的表格和顶部标题区之间间距太大了，从 24px 改成 16px。表格的行高偏高，把 el-table 的 row-style 行高从默认改成 52px。操作列的两个按钮间距太小，加 8px 的 margin-left。

这种微调很适合让 Claude Code 做。你告诉它哪里不对、怎么改，它改得又快又准，不需要你自己去翻 CSS。

2. 调色调

> 状态列的 el-tag 颜色不对。启用状态用绿色（type="success"），禁用状态用灰色（type="info"），不要用红色——禁用不是错误，不应该用 danger 色。

这是一个典型的审美判断：Claude Code 第一版可能把"禁用"设成红色（因为它觉得禁用是负面状态），但你知道在管理后台里，禁用是一个正常的状态，用灰色更合适。AI 做初稿，你做审美判断，然后让它改。

3. 调表单弹窗

> 新增提供商的弹窗宽度太宽了，600px 改成 520px。API Key 输入框改成 password 类型，加一个 "显示 / 隐藏" 切换按钮。Base URL 输入框加 placeholder 示例："https://api.openai.com/v1"。表单 label 宽度统一 100px，从左对齐改成右对齐。

4. 调响应式细节

> 当浏览器宽度小于 1200px 时，侧边栏自动折叠成只显示图标的窄模式。表格在窄屏下隐藏 Base URL 和创建时间两列，只保留关键信息。

5. 整体微调

> 整体看一下 ProviderList 页面，做以下微调：
> 1. 表格头部背景色太深了，换成 --color-bg-secondary（浅灰）
> 2. 分页器居右对齐，上方加一条细分割线
> 3. “新增提供商” 按钮加一个 Plus 图标
> 4. 鼠标悬停表格行时背景微微变色（hover 效果）
> 5. 操作列的 “编辑” 按钮用 text 类型蓝色，“删除” 用 text 类型红色，不要默认的 primary 按钮样式

你看，这一轮来回调了五六次，每次都是具体的、可执行的指令。这就是 UI 打磨的真实过程，不是一条指令出完美设计，而是先出初稿，然后你用自己的眼睛和判断来指导 Claude Code 迭代。

关键认知：**你不需要会写 CSS，但你需要能说清楚“哪里不对”和“应该怎样”。间距太大改成 16px、禁用不应该用红色用灰色、弹窗太宽改成 520px，这些描述不需要前端技能，只需要你看着屏幕，说出你的判断，Claude Code 负责把你的判断翻译成代码。**

这个细节打磨的能力，后面做每一个前端页面都会用到。Provider 页面练好了手，后面 Agent 页面、对话页面照着来就行。

验收

启动前端，打开浏览器：

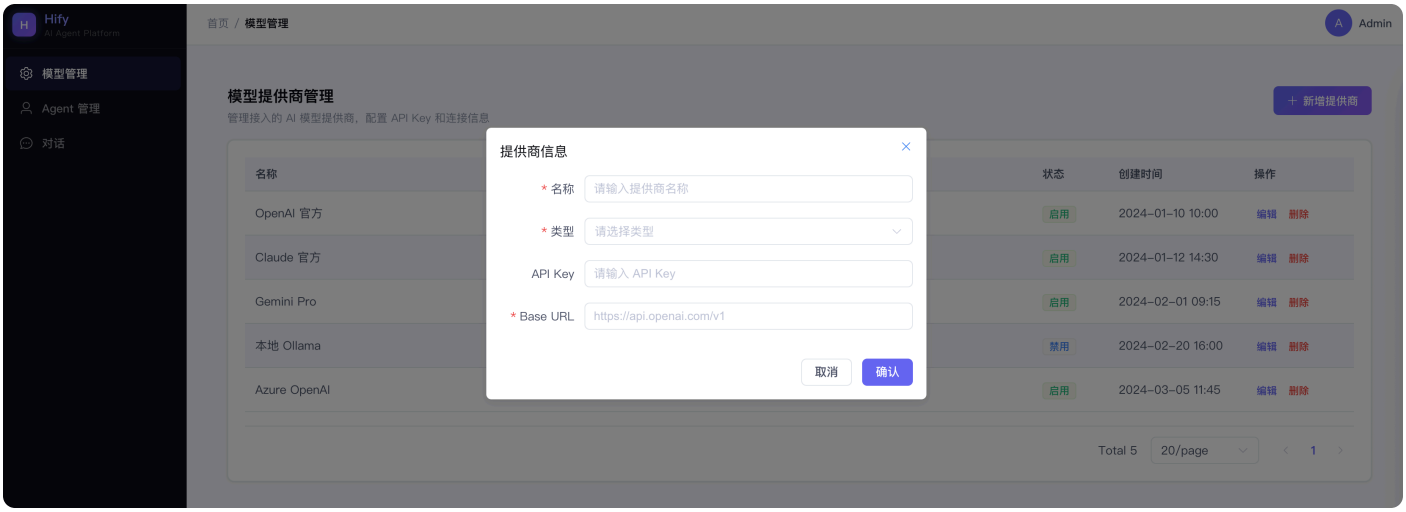
侧边栏：深色背景，Hify logo 带渐变，菜单项有图标和 hover 效果

点“模型管理”：看到 Provider 列表页，表格展示 5 条 mock 数据，分页器在底部

点“新增提供商”：弹出表单弹窗，字段完整，校验生效（name 为空提交会报错）

点某条数据的“删除”：弹出确认框，确认后提示成功，列表刷新

整体色调：浅底 + 深色侧边栏 + 蓝紫主色按钮，干净有科技感



虽然数据是 mock 的，但 UI 交互已经完整。第 13 讲做 Provider 模块时，只需要把 mock 数据换成真实 API 调用——界面不用动，只改数据源。

总结

这一讲做了三件事：让 Claude Code 当设计师搞定 UI、一条指令封装前端基础组件、来回打磨 UI 细节。

让 AI 当设计师的关键：输入要具体。不是“帮我设计好看的界面”，而是“浅底科技感、侧边栏深色、蓝紫主色、参考 Linear”。产品是什么、用户是谁、你想要什么风格、有什么参考，把这些告诉它，和给它写代码指令一样，层层递进，不让它猜。

前端组件封装：一条指令搞定。后端工程师不需要在前端细节上花太多时间。描述清楚每个组件的接口和行为，让 Claude Code 一次性生成，你验收好不好用就行。

UI 细节打磨才是真功夫。第一版永远不完美。真正的价值在于你能看着屏幕说出“哪里不对”，比如间距太大、颜色不对、弹窗太宽、hover 效果缺了。你不需要会写 CSS，但你需要有判断力。每一轮“看→说→改”都在训练这个判断力，做几个页面下来，你对 UI 的感觉会好很多。

到这里，Hify 的全部基础设施就绪了：**后端的三档组件**、**前端的设计系统**和**公共组件**、一个看起来像产品的 Provider 列表页。

从下个章节开始，我们正式进入核心功能开发。第一讲是模型提供商管理，把 mock 数据换成真实的 CRUD，对接后端 API，这是你用 Claude Code 交付的第一个完整业务模块。

思考题

找一个你觉得丑的管理后台（可以是你自己项目的，也可以是开源项目的），截个图，然后用这一讲的方法——先描述产品定位和目标风格，再让 Claude Code 给出设计系统，最后一步步改造。看看你能不能用纯文字描述把它改好看。把改造前后的对比截图分享在留言区。

期待你的分享。如果今天的课程让你有所收获，也欢迎转发给有需要的朋友，邀请他来一起学习，我们下节课再见！

© 版权归极客邦科技所有，未经许可不得传播售卖。 页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

全部留言 (11)

最新　精选



kyptonite
2026-06-11 来自重庆

可以用一下 skill /ui-ux-pro-max 把最开始的需求.md发给 cc ，配合skill让他给你出一个ui.md。然后用这个ui.md去open design上直接拿到设计图html，然后让cc在项目基础上，复原设计图的样式。

作者回复: 嘿嘿，学习到了，我不知道有这个。我试一下。感谢，技能点+1。

4



有些路，只能一个人走...
2026-05-15 来自广东

老师，你的提示词是怎么提炼出来的，能单独的教学一下吧，就比如

Hify 是一个 AI Agent 开发平台，面向技术团队内部使用，主要用户是开发者和技术管理者。界面以管理后台为主——大量的表格、表单、配置页面，加上一个对话交互页面。我想要的视觉风格：浅底 + 科技感点缀。整体用浅色背景保持信息可读性（管理后台表格多，深色底长时间看眼睛累）。但不要太素——侧边栏用深色底，按钮和关键交互元素用亮色，制造科技感和品牌感。色调方向：主色用蓝紫系（科技感强），辅色用青色或薄荷绿（数据 / 状态指示）。参考 Linear、Supabase 的视觉风格——干净但不无聊，有设计感但不花哨。帮我设计一套完整的设计系统：主色 / 辅色 / 背景色阶 / 文字色阶 / 圆角 / 阴影 / 过渡动效，用 CSS 变量输出。

这些风格类的，你是如何喂给ai提炼出这些提示词的

> 作者回复: 这些提示词，有两种思路：
> 1. 你把这些提示词复制，自己改吧改吧，就变成自己的经验了，以后就会写了。
> 2. 让AI 生成这些提示词的话。我一般就是告诉它“我要科技感的风格，你给我几个可以选”。然后根据提示对话下去。生成后如果不满意，那就继续换。
>
> 我一开始的提示词都比较简单。比如上面那句话。然后经过几轮的对话调整，让AI直接总结提示词。




wahaha13168
2026-05-14 来自广东

没有美感的程序员怎么办

> 作者回复: 你可以问cc，有什么风格选。然后你选一个。或者一个一个实验下来，应该就有你喜欢的了。我都是这么搞的。
>
> 或者你找个截图，扔给ta，告诉cc，我就要这个风格和布局。
>
> 没有美感，你应该有你喜欢的颜色，告诉ta色系就可以了






霰雪纷飞
2026-04-24 来自上海

这里用了大量的专业名词，比如用 HifyTable 和 HifyFormDialog 实现XXX，所有组件使用 Vue 3 Composition API + TypeScript + Element Plus，没有点前端基础，根本都用不上这个。。。。

作者回复: 这个是我让AI 给我推荐的。为了教学后，大家执行的效果写的比较细。– –

2

chenjunjiang
2026-04-15 来自四川

看到现在，有个感受：老师完全是站在技术专家的角度来和Claude Code进行对话的，很多专业术语对于非技术同学来说根本看不懂，技术是积累的过程，不可能短时间内就能搞懂的。当然，我自己是技术出身，倒是没啥问题。我认为Claude Code应该作为乙方，我是甲方，我把需求给你描述清楚了，就就按最佳实践来就行，而不是让甲方还要把技术细节给你讲清楚。

作者回复: 因为这门课面向的群体是技术出身的同学。你说的这个视角是纯产品视角和甲方视角。
这门课的主要思考路径是：我是一个有技术经验的研发，然后怎么和cc配合做出一款产品。

2



迪马
2026-04-14 来自广东

产品不能等你整个开发出来才演示吧，传统的原型用什么来替代

作者回复: 产品经理有产品经理的专业原型工具。我自己的话，野路子用：https://excalidraw.com/

2



s(t)
2026-07-02 来自上海

别说后端写不出来了，我前端也写不出来这些提示词啊哈哈哈

作者回复: 所以，要学会让ai帮你写。哈哈哈。跟AI讨论，肯定不能自己写。

1



风轻扬

2026-05-28 来自北京

看到评论区我就放心了，不只是我自己有疑问。这一连串专业前端名词，真心看不懂。如果是借助cc生成的，可以把生成过程简单提下，这样就更好入门点

作者回复: 在另外的评论已回答～

1



-m

2026-04-14 来自广东

老师，从提示词不难看出你是会一点前端技能的，对于一个真正没有接触过前端的人来说，像你这种提示词，真的写不出来，更多的是让ai去完善提示词，想了解下你这些提示词是否有和ai讨论过呢

作者回复: 我的真实情况是。

1. 我是知道现在前端的主流框架的，比如vue，react等等。然后最近发现有一个叫vitpress。
2. 我仅仅知道这两个框架和词而已。技术细节没去学过，其他我也就不懂了。

我的提示词这么细是因为我让cc教了我一下。然后为了教学效果写的细了一点。

其实我认为，知道vue + vitpress，告诉cc： 用vue + vitpress给我搭建一个后台款系统前端。其实就能出来一个，但是就得不断的去调，达到自己的预期。

共 3 条评论>

1



Joe

2026-07-05 来自广东

伪全栈是这样的, 看前端代码so easy, 看java代码(这个组件干嘛的? 这个注释干啥用? 为什么用这个包?), 还都需要沉淀,,,

作者回复: 哈哈哈，后端确实内容比较多，以后大家全部都是全栈了。我是后端，然后感觉我使用AI也能做很不错的前端。
卷起来啊，大兄弟，搞ta～